# Trabajo práctico: Listas enlazadas simples con Java y OpenCode

## Consigna general

Implementar en Java los siguientes ejercicios utilizando **OpenCode** como asistente de desarrollo.

Para cada ejercicio, el alumno deberá escribir un prompt inicial donde explique el problema, el razonamiento esperado, los casos a contemplar y la estructura de clases que considera necesaria.

El objetivo no es solamente que el código funcione, sino que el prompt demuestre que el alumno entendió la lógica de las listas enlazadas simples: nodos, referencias, `head`, recorrido secuencial, inserción, eliminación y actualización de enlaces.

En el código entregado deberán incluir como comentario:

/*
Prompt inicial utilizado:
...

Ajustes realizados luego de la primera respuesta de OpenCode:
...
*/

---

# Ejercicio 1: Crear una lista enlazada simple desde cero

Implementar una lista enlazada simple de números enteros, sin utilizar estructuras de Java como `ArrayList`, `LinkedList` o similares.

La lista debe permitir:

insertarAlInicio(int dato)
insertarAlFinal(int dato)
imprimir()
estaVacia()
getSize()

El prompt debe explicar cómo se representa un nodo, qué función cumple `head` y cómo se actualiza el tamaño de la lista.

---

# Ejercicio 2: Buscar elementos en una lista enlazada

A partir de una lista enlazada simple de enteros, implementar un método:

boolean buscar(int dato)

El método debe recorrer la lista desde `head` hasta encontrar el dato o llegar a `null`.

El prompt debe explicar por qué no se puede acceder directamente por índice y por qué la búsqueda requiere recorrido secuencial.

---

# Ejercicio 3: Obtener un elemento por posición

Implementar el método:

int obtener(int posicion)

El método debe devolver el dato almacenado en una posición determinada.

Debe validar posiciones inválidas:

posicion < 0
posicion >= size

El prompt debe explicar que, aunque se use una posición, la lista enlazada no tiene acceso directo como un arreglo, por lo tanto debe recorrerse nodo por nodo.

---

# Ejercicio 4: Insertar un nodo en una posición específica

Implementar el método:

void insertarEnPosicion(int dato, int posicion)

Debe permitir insertar:

- al inicio
- en el medio
- al final

Debe validar posiciones inválidas.

El prompt debe explicar con claridad el orden correcto de actualización de referencias:

nuevo.setSiguiente(actual.getSiguiente());
actual.setSiguiente(nuevo);

También debe explicar qué podría pasar si se actualizan las referencias en el orden incorrecto.

---

# Ejercicio 5: Eliminar un nodo por valor

Implementar el método:

boolean eliminar(int dato)

Debe eliminar la primera aparición del dato en la lista.

Debe contemplar:

- lista vacía
- eliminar el primer nodo
- eliminar un nodo del medio
- eliminar el último nodo
- dato inexistente

El prompt debe explicar que en Java no se borra manualmente el nodo, sino que se deja inaccesible para que luego pueda actuar el Garbage Collector.

---

# Ejercicio 6: Eliminar un nodo por posición

Implementar el método:

void eliminarEnPosicion(int posicion)

Debe validar posiciones inválidas y actualizar correctamente el atributo `size`.

El prompt debe explicar cómo encontrar el nodo anterior al que se quiere eliminar y cómo hacer que apunte al nodo siguiente del eliminado.

---

# Ejercicio 7: Modificar un elemento de la lista

Implementar el método:

void modificar(int posicion, int nuevoDato)

Debe cambiar el dato almacenado en una posición determinada.

Debe validar la posición antes de recorrer.

El prompt debe explicar la diferencia entre modificar el dato de un nodo y modificar la referencia al siguiente nodo.

---

# Ejercicio 8: Contar ocurrencias de un valor

Implementar el método:

int contarOcurrencias(int dato)

El método debe recorrer toda la lista y contar cuántas veces aparece un valor.

Ejemplo:

10 -> 20 -> 10 -> 30 -> 10 -> null

Para el valor `10`, debe devolver:

3

El prompt debe explicar por qué no se debe cortar el recorrido al encontrar la primera coincidencia.

---

# Ejercicio 9: Invertir una lista enlazada simple

Implementar el método:

void invertir()

Ejemplo:

10 -> 20 -> 30 -> 40 -> null

Después de invertir:

40 -> 30 -> 20 -> 10 -> null

El prompt debe explicar qué referencias auxiliares hacen falta para invertir la lista sin perder nodos.

Por ejemplo:

anterior
actual
siguiente

También debe explicar por qué el orden de actualización de referencias es fundamental.

---

# Ejercicio 10: Convertir la lista de enteros en una lista genérica

Tomar la implementación de lista enlazada simple con `int` y transformarla en una versión genérica:

ListaEnlazada<T>
Nodo<T>

Debe funcionar con distintos tipos de datos, por ejemplo:

ListaEnlazada<Integer>

ListaEnlazada<String>
ListaEnlazada<Alumno>

El prompt debe explicar qué partes del código cambian, cuáles se mantienen iguales y por qué el algoritmo de recorrido, inserción y eliminación no depende del tipo de dato almacenado.